EXPERIMENT 1

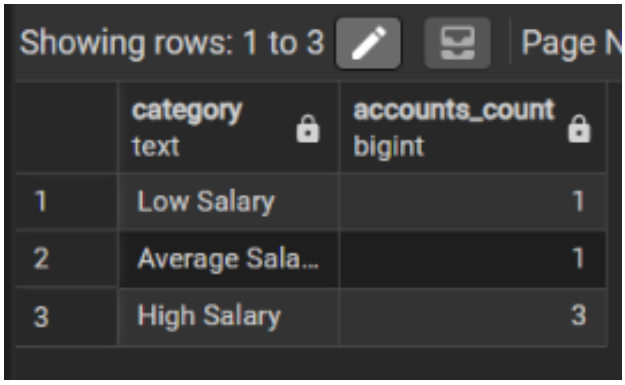
(A)

Write an SQL solution to calculate the total number of bank accounts belonging to each salary category. The classification categories are:

1. **"Low Salary"**: All monthly salaries strictly less than $20,000.
2. **"Average Salary"**: All monthly salaries in the inclusive range [$20,000, $50,000].
3. **"High Salary"**: All monthly salaries strictly greater than $50,000. The final result table must report all three categories, even if a category contains zero accounts.

CODE:

```
CREATE TABLE Accounts (
   aid INT PRIMARY KEY,
   income INT NOT NULL);
INSERT INTO Accounts (aid, income) VALUES
(3, 108939),
(2, 12747),
(8, 87709),
(6, 91738),
(7, 45169);
SELECT 'Low Salary' AS category, COUNT(aid) AS accounts_count
FROM Accounts
WHERE income < 20000
UNION ALL
SELECT 'Average Salary' AS category, COUNT(aid) AS accounts_count
FROM Accounts
WHERE income >= 20000 AND income <= 50000
UNION ALL
SELECT 'High Salary' AS category, COUNT(aid) AS accounts_count FROM Accounts
WHERE income > 50000;
```


Showing rows: 1 to 3 Page N

	category text	accounts_count bigint
1	Low Salary	1
2	Average Sala...	1
3	High Salary	3

(B)

Design an Employee Management System database architecture by implementing the following operations sequentially:

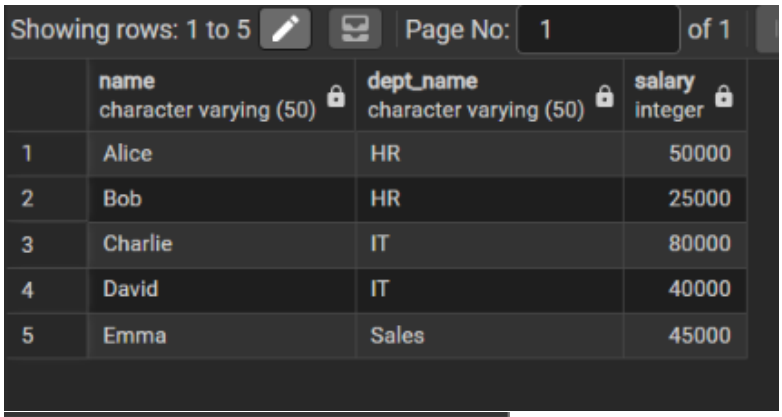
1. Create and populate structural tracking tables for Departments, Employees, and Salaries.
2. Query the system using multi-table Joins to pull clear department breakdowns, apply a 10% salary enhancement across all members of the HR department, and delete any individual salary record dropping strictly below $30,000.

List all remaining active employees whose personal salary exceeds the overall company-wide average salary.

CODE:

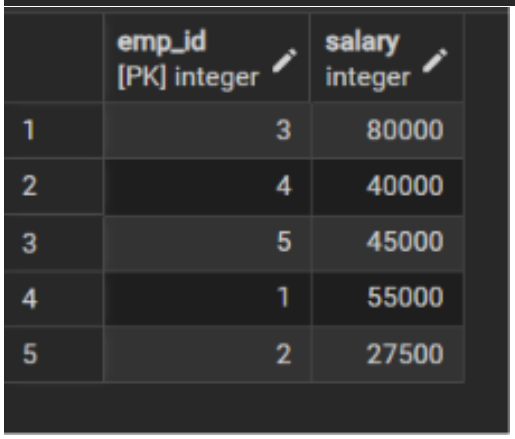
```
SELECT e.name, d.dept_name, s.salary
FROM Employees AS e JOIN Departments AS d
ON e.dept_id = d.dept_id
JOIN Salaries AS s ON e.emp_id = s.emp_id;


UPDATE Salaries SET salary = salary * 1.10
WHERE emp_id IN (
   SELECT e.emp_id
        FROM Employees e  JOIN Departments d
        ON e.dept_id = d.dept_id
        WHERE d.dept_name = 'HR');


select * from Salaries;
SELECT e.name, s.salary
```

```
FROM Employees AS e
JOIN Salaries AS s
ON e.emp_id = s.emp_id
WHERE s.salary > (SELECT AVG(salary) FROM Salaries);
```

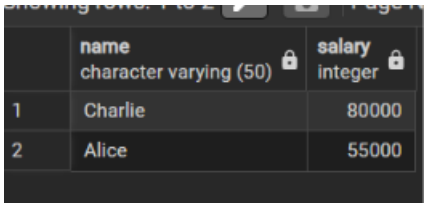


Showing rows: 1 to 5 Page No: 1 of 1

	name character varying (50)	dept_name character varying (50)	salary integer
1	Alice	HR	50000
2	Bob	HR	25000
3	Charlie	IT	80000
4	David	IT	40000
5	Emma	Sales	45000



	emp_id [PK] integer	salary integer
1	3	80000
2	4	40000
3	5	45000
4	1	55000
5	2	27500



	name character varying (50)	salary integer
1	Charlie	80000
2	Alice	55000

(C)

A pizza restaurant stores all the available pizza toppings and their ingredient costs in a table called pizza_toppings.

Every customer must choose **exactly 3 different toppings** to prepare a pizza.

Write a PostgreSQL query to generate **every possible 3-topping pizza combination** along with its **total ingredient cost**.

Requirements

Every pizza must contain **exactly 3 different toppings**.

The same topping **cannot appear more than once** in a pizza.

Different arrangements of the same toppings should be considered the **same pizza**.

Example:

Chicken, Pepperoni, Sausage

Pepperoni, Chicken, Sausage

Sausage, Chicken, Pepperoni

These are all the same pizza, so only **one** should appear.

Display the topping names separated by commas.

Display the total ingredient cost of each pizza.

Sort the output:

First by **highest total cost**

If two pizzas have the same cost, sort them alphabetically.

CODE:

```
SELECT
    -- 1. Concatenate the three distinct topping names with commas
    CONCAT(t1.topping_name, ',', t2.topping_name, ',', t3.topping_name) AS pizza,
    -- 2. Sum the individual ingredient costs for the total cost
    (t1.ingredient_cost + t2.ingredient_cost + t3.ingredient_cost) AS total_cost
FROM pizza_toppings t1
-- Join a second instance of the table where alphabetical order is enforced
INNER JOIN pizza_toppings t2
    ON t1.topping_name < t2.topping_name
-- Join a third instance of the table maintaining the alphabetical order chain
INNER JOIN pizza_toppings t3
    ON t2.topping_name < t3.topping_name
-- Sort first by highest total cost, then alphabetically by the pizza string
ORDER BY
    total_cost DESC,
    pizza ASC;
```

	pizza text	total_cost numeric
1	Chicken,Pepperoni,Sausage	1.75
2	Chicken,Extra Cheese,Sausage	1.65
3	Extra Cheese,Pepperoni,Saus...	1.60
4	Chicken,Onions,Sausage	1.50
5	Chicken,Extra Cheese,Pepper...	1.45
6	Onions,Pepperoni,Sausage	1.45
7	Extra Cheese,Onions,Sausage	1.35
8	Chicken,Onions,Pepperoni	1.30
9	Chicken,Extra Cheese,Onions	1.20
10	Extra Cheese,Onions,Pepperoni	1.15

(D)

Amazon wants to identify **returning customers** who made another purchase shortly after their first purchase.

Write a PostgreSQL query to find all users who made **another purchase within 1 to 7 days** of an earlier purchase.

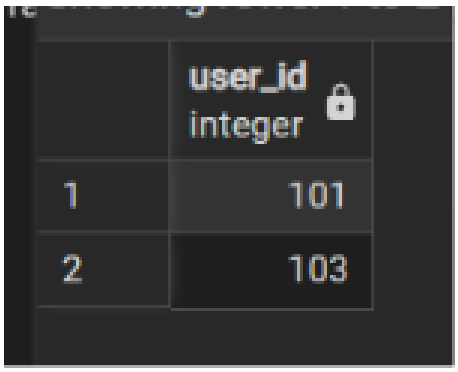
Requirements

1. Compare purchases **made by the same user only**.
2. Ignore comparisons where the same transaction is matched with itself.
3. The second purchase must occur **after** the first purchase.
4. Same-day purchases must **not** be considered.
5. The second purchase must occur within **7 days** of the first purchase.
6. Display only the **unique User IDs** that satisfy the above conditions.
7. Sort the output in ascending order of user_id.

CODE:

```
SELECT DISTINCT t1.user_id

FROM amazon_transactions t1

INNER JOIN amazon_transactions t2

   ON t1.user_id = t2.user_id
```

```
WHERE
    -- Ensures the second transaction happens on a later day (ignores same-day/same-id matches)
    t2.created_at::date > t1.created_at::date
    -- Restricts the second transaction to occur within 7 days of the first transaction
    AND t2.created_at::date <= t1.created_at::date + INTERVAL '7 days'
ORDER BY t1.user_id ASC;
```



	user_id integer
1	101
2	103

24BAI70068

24BAI70068

24BAI70068